

100 Python Interview Questions — Actually Asked

💡 Real questions from real interviews at companies like Google, Amazon, Meta, Deloitte, and startups. Answers are direct and human — not textbook definitions.

🟦 Basics — Q01 to Q20

🎯 Every Python interview starts here. Get these cold before anything else.

Q01 — What is the difference between a list and a tuple?

Lists are mutable — you can change them after creation. Tuples are immutable — once created, that's it. Tuples are slightly faster and are used when the data shouldn't change: coordinates, RGB values, function return pairs.

Q02 — What does `*args` and `**kwargs` do?

`*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. You use them when you don't know in advance how many arguments a function will receive.

```
def show(*args, **kwargs):  
    print(args)    # tuple  
    print(kwargs)  # dict  
show(1, 2, name='Alice')  # (1, 2), {'name': 'Alice'}
```

Q03 — What is the difference between `==` and `is`?

`==` checks if two values are equal. `is` checks if they're the same object in memory. Two variables can be `==` without being `is`. Small integers (-5 to 256) and short strings are cached so `is` might return True — but never rely on that.

Q04 — What are mutable and immutable types in Python?

Mutable: list, dict, set, bytearray. Immutable: int, float, str, tuple, bool, frozenset. Immutable objects can be used as dict keys. Mutable objects passed to a function can be modified inside it — that surprises people.

Q05 — What is a list comprehension and when do you use it?

A compact way to build a list: `[expr for item in iterable if condition]`. Use it when the logic is simple enough to read in one line. When there's exception handling or 3+ conditions, use a regular loop.

Q06 — What is the difference between `append()` and `extend()`?

`append()` adds one item to the end. `extend()` adds every item from an iterable. `lst.append([1,2])` makes the list one longer. `lst.extend([1,2])` makes it two longer.

Q07 — How does Python handle memory management?

Python uses reference counting plus a garbage collector for cyclic references. When an object's reference count drops to zero it's freed. The `gc` module handles circular references the reference counter can't detect.

Q08 — What is a lambda function?

An anonymous function written in one line: `lambda args: expression`. The expression is always returned — no `return` needed. Best inside `sorted(key=...)`, `map()`, `filter()`. Never assign a lambda to a variable — use `def` for that.

Q09 — What is the difference between `deepcopy` and `copy`?

`copy.copy()` is a shallow copy — it copies the outer object but nested objects are still shared. `copy.deepcopy()` copies everything recursively. For a list of lists, shallow copy means modifying a nested list affects both copies.

Q10 — What does `if __name__ == '__main__':` do?

Python sets `__name__` to `'__main__'` only when the file is run directly. When the file is imported, `__name__` is the module name. This pattern lets you write code that runs when executed directly but not when imported as a module.

Q11 — What is the GIL and why does it matter?

The Global Interpreter Lock is a mutex that prevents multiple threads from executing Python bytecode simultaneously. This means Python threads don't give you true CPU parallelism for CPU-bound tasks. For CPU work, use `multiprocessing`. For I/O-bound work (network, disk), threads are fine because they release the GIL while waiting.

Q12 — What is slicing and how does it work?

`lst[start:stop:step]` — start is inclusive, stop is exclusive. `lst[::-1]` reverses. `lst[::2]` every other element. Negative indices count from the end. Slicing never raises `IndexError` — if you go out of bounds it just returns what's there.

Q13 — What are Python's built-in data types?

int, float, complex, bool, str, bytes, list, tuple, set, frozenset, dict, `NoneType`. Everything in Python is an object, including functions and classes.

Q14 — What is the difference between `range()` and `xrange()` ?

In Python 3, `xrange()` doesn't exist — `range()` is already lazy (returns a range object, not a list). In Python 2, `range()` returned a list and `xrange()` was the lazy version. If you're asked this, clarify which Python version and note that Python 2 is end-of-life.

Q15 — How do you reverse a list in Python?

Three ways: `lst.reverse()` in place, `lst[::-1]` returns a new reversed list, `list(reversed(lst))` uses an iterator. The interviewer usually wants you to know all three and explain the difference.

Q16 — What is the difference between `remove()`, `pop()`, and `del` ?

`remove(val)` removes the first occurrence of a value. `pop(i)` removes by index and returns the element. `del lst[i]` removes by index, returns nothing. `pop()` with no argument removes the last element.

Q17 — What is string interning?

Python automatically reuses the same string object for short strings and identifiers. So `a = 'hello'; b = 'hello'; a is b` is `True`. Not guaranteed for all strings — longer strings or strings with spaces may not be interned. Use `==` for string comparison, never `is`.

Q18 — What are Python's comparison operators and how does chaining work?

Python allows chaining: `1 < x < 10` is valid and means `1 < x and x < 10`. Most languages don't support this. It only evaluates `x` once.

Q19 — What is the difference between `str()` and `repr()` ?

`str()` is for human-readable output. `repr()` is for developer-readable output — it should ideally return something that could recreate the object. `repr()` is what

Python shows in the console. Override `__str__` and `__repr__` in your own classes.

Q20 — How do you swap two variables in Python?

`a, b = b, a` — Python evaluates the right side first as a tuple, then unpacks. No temp variable needed. This is the expected answer in interviews.

🟡 Functions & Scope — Q21 to Q35

🎯 Scope and closures trip up most candidates. These questions separate people who understand Python from those who just use it.

Q21 — What is the LEGB rule?

Python looks up names in this order: **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in. If a name isn't found at one level, it goes up. This is Python's scoping rule — understanding it explains most "variable not defined" errors.

Q22 — What is a closure?

A function that remembers the variables from its enclosing scope even after the outer function has finished. The inner function "closes over" those variables.

```
def make_multiplier(n):
    def multiply(x):
        return x * n    # n is from enclosing scope
    return multiply

double = make_multiplier(2)
print(double(5))    # 10 — n=2 is remembered
```

Q23 — What is the mutable default argument trap?

Default arguments are evaluated once when the function is defined, not each time it's called. If you use a mutable default like `[]` or `{}` , it persists between calls.

```
# Wrong
def add(item, lst=[]):
    lst.append(item)
```

```

    return lst

# Right
def add(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst

```

Q24 — What does `global` and `nonlocal` do?

`global` lets a function read AND write a global variable. Without it, assigning to a name inside a function creates a local variable. `nonlocal` does the same for enclosing (not global) scope — useful in closures.

Q25 — What is a decorator?

A function that wraps another function to extend or modify its behaviour without changing the original. The `@` syntax is just shorthand: `@my_decorator` on top of `def f():` is the same as `f = my_decorator(f)`.

```

def log(func):
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__}')
        return func(*args, **kwargs)
    return wrapper

@log
def greet(name):
    return f'Hi {name}'

```

Q26 — What is a generator and how is it different from a list?

A generator produces values one at a time using `yield`. It doesn't store the whole sequence in memory — values are computed on demand. `{x**2 for x in range(million)}` uses constant memory. `[x**2 for x in range(million)]` uses 8MB.

Q27 — What does `yield` do?

Pauses the function and returns a value to the caller. When the generator is called again, it resumes from where it left off. A function with `yield` is a

generator function — calling it returns a generator object.

Q28 — What is `functools.wraps` and why use it?

When you wrap a function with a decorator, the wrapper replaces the original's `__name__`, `__doc__`, etc. `@functools.wraps(func)` preserves those attributes so debugging tools and docs still show the original function's name.

Q29 — What is the difference between a function and a method?

A method is a function defined inside a class that receives `self` as its first argument. All methods are functions — not all functions are methods.

Q30 — What does `map()` return in Python 3?

A map object — a lazy iterator, not a list. Wrap with `list()` to see results. This is a common interview gotcha: `map(func, lst)` doesn't immediately execute anything.

Q31 — What is `filter()` and how is it different from a list comprehension?

`filter(fn, lst)` returns elements where `fn(element)` is True. Equivalent to `[x for x in lst if fn(x)]`. Both are fine — comprehensions are usually more readable. `filter(None, lst)` removes falsy values.

Q32 — What does `sorted()` return vs `.sort()`?

`sorted()` returns a new sorted list, leaving the original unchanged. `.sort()` sorts in place and returns None. Classic mistake: `lst = lst.sort()` gives you None.

Q33 — What is partial application with `functools.partial`?

Creates a new function with some arguments pre-filled.

```
from functools import partial
def power(base, exp): return base ** exp
square = partial(power, exp=2)
print(square(5)) # 25
```

Q34 — How does Python's `zip()` work?

Pairs elements from multiple iterables. Stops at the shortest. `dict(zip(keys, vals))` is one of the most used patterns. `zip(*matrix)` transposes a 2D list.

Q35 — What is `enumerate()` and when do you use it?

Wraps an iterable and returns (index, value) pairs. Always use it instead of `range(len(lst))` when you need both. `enumerate(lst, start=1)` starts counting from 1.

● Data Structures — Q36 to Q50

🕒 Dict internals, sets, and complexity questions appear in almost every technical screen.

Q36 — How does a Python dictionary work internally?

Dicts use a hash table. Keys are hashed and the hash determines where the value is stored. Lookup is $O(1)$ average case. Keys must be hashable (immutable). Since Python 3.7, dicts preserve insertion order.

Q37 — What makes an object hashable?

An object is hashable if it has a `__hash__` method that returns the same value for its lifetime, and a `__eq__` method. Immutable types are hashable. Mutable types (list, dict, set) are not — you can't use them as dict keys.

Q38 — What is a set and when do you use it?

An unordered collection of unique hashable elements. Membership testing is $O(1)$. Use sets for: deduplication `list(set(lst))`, membership checks `x in my_set`, and set operations (union, intersection, difference).

Q39 — What is the difference between a set and a frozenset?

A frozenset is an immutable set. It can be used as a dict key or as an element in another set. Regular sets cannot.

Q40 — What is a defaultdict?

From `collections`. Like a regular dict but returns a default value for missing keys instead of raising `KeyError`. `defaultdict(list)` returns `[]` for missing keys, `defaultdict(int)` returns 0.

```
from collections import defaultdict
d = defaultdict(list)
d['key'].append(1) # no KeyError
```

Q41 — What is a Counter?

From `collections`. Counts hashable objects. `Counter('aabbabc')` gives `{'a':3, 'b':2, 'c':1}`. `.most_common(n)` returns top N. Useful for frequency analysis, anagram detection.

Q42 — What is a deque?

From `collections`. Double-ended queue. Append and pop from both ends in $O(1)$. Lists are $O(n)$ for operations at the front. Use deque for queues, sliding windows, and BFS.

Q43 — What is the time complexity of common list operations?

Operation	Complexity
append	$O(1)$ amortized
insert at index	$O(n)$
pop (end)	$O(1)$
pop (index)	$O(n)$
in operator	$O(n)$
index lookup	$O(1)$

Q44 — What is the time complexity of dict and set operations?

Lookup, insert, delete: $O(1)$ average, $O(n)$ worst case (hash collision). In practice always treat as $O(1)$. The `in` operator on a set is $O(1)$ vs $O(n)$ for a list — this matters at scale.

Q45 — How do you merge two dicts?

Python 3.9+: `merged = d1 | d2`. Before 3.9: `merged = {**d1, **d2}`. Both create a new dict. `d1.update(d2)` modifies d1 in place. In case of duplicate keys, the second dict wins.

Q46 — What is dict comprehension?

`{k: v for k, v in items}` — builds a dict in one line. `{v: k for k, v in d.items()}` flips a dict. `{k: v for k, v in d.items() if v > 0}` filters.

Q47 — What is the difference between `list()` and `[]`?

`[]` is a literal — faster. `list()` calls the constructor — can convert any iterable. `list('abc')` gives `['a', 'b', 'c']`. For creating empty lists, always use `[]`.

Q48 — How do you sort a list of dicts by a key?

`sorted(lst, key=lambda d: d['field'])` — don't need to import anything. `reverse=True` for descending. `key=lambda d: (d['a'], d['b'])` for multi-field sort.

Q49 — What is `heapq` used for?

Python's heap implementation — a min-heap. `heapq.nlargest(n, lst)` and `heapq.nsmallest(n, lst)` are faster than sorting the whole list when n is small. Used in Dijkstra's algorithm.

Q50 — What is `bisect` used for?

Binary search on sorted lists. `bisect.bisect_left(lst, x)` finds the insertion point. $O(\log n)$. Use when you need to search or insert into a sorted list efficiently.

OOP — Q51 to Q65

🎯 OOP questions test depth. Interviewers want to see that you understand why these concepts exist, not just the syntax.

Q51 — What are the four pillars of OOP?

Encapsulation (bundle data + methods, control access), Inheritance (child gets parent's attributes + can add/override), Polymorphism (same method name, different behaviour per class), Abstraction (hide complexity, expose interface).

Q52 — What is the difference between `__str__` and `__repr__`?

`__str__` is for humans — what `print()` shows. `__repr__` is for developers — what the console shows, ideally something that could recreate the object. If only `__repr__` is defined, Python uses it for both.

Q53 — What is `super()` and why is it needed?

`super()` calls the parent class's method. Always call `super().__init__()` in a child class — without it the parent's setup never runs, causing `AttributeErrors`. In multiple inheritance, `super()` follows the MRO.

Q54 — What is the MRO (Method Resolution Order)?

The order Python searches for methods in a class hierarchy. Use `ClassName.__mro__` or `ClassName.mro()` to inspect it. Python uses the C3 linearization algorithm. This matters most with multiple inheritance.

Q55 — What is the difference between class, static, and instance methods?

Instance methods receive `self` — access instance + class data. Class methods receive `cls` — access class data only, decorated with `@classmethod`. Static methods receive neither — utility functions, decorated with `@staticmethod`.

Q56 — What is `@property`?

Makes a method look like an attribute. Lets you add validation or computation to attribute access without changing how callers use the class. Pair with `@attr.setter` for controlled writes.

Q57 — What is the difference between `_name`, `__name`, and `__name__`?

`_name` — convention: protected, don't access outside class. `__name` — name-mangled by Python to `_ClassName_name`, harder to access accidentally. `__name__` — dunder attribute, part of Python's protocol.

Q58 — What is an abstract class?

A class that can't be instantiated directly. Forces subclasses to implement required methods. Use `from abc import ABC, abstractmethod`. If a subclass doesn't implement all abstract methods, instantiating it raises `TypeError`.

Q59 — What is duck typing?

"If it walks like a duck and quacks like a duck, it's a duck." Python doesn't check types — it checks whether an object has the required method or attribute. This is why `len()` works on strings, lists, dicts, and custom objects as long as they implement `__len__`.

Q60 — What is a dataclass?

Decorator from `dataclasses`. Automatically generates `__init__`, `__repr__`, `__eq__` based on field annotations. Less boilerplate for data-container classes. `frozen=True` makes it immutable.

Q61 — What is `__slots__`?

Replaces the per-instance `__dict__` with a fixed set of attributes. Reduces memory usage significantly for classes with many instances. Trade-off: you can't add arbitrary attributes at runtime.

Q62 — What is the difference between aggregation and composition?

Composition: the contained object can't exist without the container.
Aggregation: it can. A `Car` composed of an `Engine` — destroy the car, the

engine is destroyed. A `Team` aggregating `Players` — dissolve the team, players still exist.

Q63 — What is a context manager and how do you write one?

Implement `__enter__` and `__exit__` on a class, then use it with `with`. `__enter__` sets up, `__exit__` tears down (even if an exception occurs). Or use `@contextmanager` from `contextlib` with `yield`.

Q64 — What is multiple inheritance and what problem does it cause?

A class can inherit from multiple parents. The diamond problem: if two parents both inherit from the same grandparent, which version of the grandparent's method runs? Python resolves this with MRO. `super()` follows MRO so each class in the chain is called once.

Q65 — What is the difference between `isinstance()` and `type()` ?

`isinstance(obj, ClassName)` returns True for the class and any subclass. `type(obj) == ClassName` is strict — only True for that exact class. Always prefer `isinstance()` because it respects inheritance.

Error Handling & File I/O — Q66 to Q75

Q66 — What is the structure of try/except/else/finally?

`try` — code that might fail. `except` — runs if exception occurs. `else` — runs if NO exception occurred. `finally` — always runs, exception or not. Use `else` for code that should only run on success — cleaner than putting it in `try`.

Q67 — What is the difference between `Exception` and `BaseException` ?

`Exception` is the base for most user-facing exceptions. `BaseException` also includes `SystemExit`, `KeyboardInterrupt`, `GeneratorExit` — things you usually don't want to catch. Always catch `Exception`, not `BaseException`, unless you specifically need those.

Q68 — What happens if you use bare `except:` ?

It catches everything including `KeyboardInterrupt` and `SystemExit` — your program can't be stopped with Ctrl+C. Always use `except Exception:` at minimum, or better, catch specific exceptions.

Q69 — How do you raise a custom exception?

```
class ValidationError(Exception):
    pass

raise ValidationError('Amount must be positive')
```

Subclass `Exception`. You can add `__init__` to pass custom data. Use `raise ... from err` to chain exceptions and preserve the original traceback.

Q70 — What is `with open()` and why use it?

Context manager for file operations. Automatically closes the file when the block exits — even if an exception occurs. Never use raw `open()` without `with`. `with open('f.txt') as f:` is the right pattern.

Q71 — What is the difference between `read()`, `readline()`, and `readlines()`?

`read()` — entire file as one string. `readline()` — one line at a time. `readlines()` — all lines as a list. For large files, iterate the file object directly: `for line in f:` — memory efficient.

Q72 — What file modes are available in `open()`?

`'r'` read, `'w'` write (overwrites), `'a'` append, `'rb'` / `'wb'` binary, `'r+'` read+write. Default is `'r'`. `'w'` creates the file if it doesn't exist and wipes it if it does.

Q73 — How do you handle encoding issues when reading files?

Always specify `encoding='utf-8'` explicitly: `open('f.txt', encoding='utf-8')`. Without it, Python uses the system default which varies by OS. Use `errors='ignore'` or `errors='replace'` if the file has bad characters.

Q74 — What is `pathlib` and why is it better than `os.path`?

`pathlib.Path` represents file paths as objects. `path / 'subdir' / 'file.txt'` instead of `os.path.join()`. Methods like `.exists()`, `.read_text()`, `.write_text()`, `.suffix`, `.stem` are all built in. Much cleaner than string-based path manipulation.

Q75 — What does `assert` do and when should you NOT use it?

`assert condition, 'message'` raises `AssertionError` if condition is False. Use for development-time checks and test assertions. Never use for input validation or

business logic in production — `assert` statements are disabled when Python runs with the `-O` flag.

● pandas & Data — Q76 to Q88

🎯 These come up in every data analyst/scientist role. Real questions about real daily use.

Q76 — What is the difference between `loc` and `iloc`?

`loc` is label-based — inclusive on both ends. `iloc` is position-based — exclusive on the right (like Python slicing). `df.loc[0:5]` returns 6 rows. `df.iloc[0:5]` returns 5.

Q77 — What does `axis=0` vs `axis=1` mean in pandas?

`axis=0` operates down rows — gives one value per column. `axis=1` operates across columns — gives one value per row. `df.drop('col', axis=1)` drops a column. `df.sum(axis=0)` sums each column.

Q78 — What is the difference between `apply()` and `map()` in pandas?

`map()` works on a Series, element by element. `apply()` works on Series or DataFrame — `df.apply(fn, axis=1)` applies row-wise. `applymap()` (now `map()` in newer pandas) applies element-wise on a DataFrame.

Q79 — How do you handle missing values in pandas?

`df.isnull().sum()` to count. `dropna()` to remove rows/columns. `fillna(value)` to replace — can use `method='ffill'` or `method='bfill'`. For numeric columns, fill with median: `df['col'].fillna(df['col'].median())`.

Q80 — What is `groupby()` and how does named aggregation work?

`groupby()` groups rows and applies aggregation. Named aggregation in `agg()` gives clean column names: `df.groupby('region').agg(total=('revenue', 'sum'), avg=('revenue', 'mean'))`. No renaming step needed.

Q81 — What is the difference between `merge()` and `concat()`?

`merge()` is SQL-style JOIN — combines on a key column. `concat()` stacks DataFrames vertically (`axis=0`) or horizontally (`axis=1`). Always use `ignore_index=True` with `concat` to reset index.

Q82 — What does `pd.melt()` do?

Converts wide format to long format. Turns column headers into a column of values. `id_vars` are the columns to keep as-is. `value_vars` are the columns to unpivot.

Q83 — What is vectorisation in NumPy/pandas?

Applying an operation to the whole array at once instead of looping. `arr * 2` multiplies every element — no Python loop. 10-100x faster than equivalent loops because operations are implemented in C. Always prefer vectorised operations over `iterrows()`.

Q84 — What is broadcasting in NumPy?

Applying operations between arrays of different shapes without loops. `arr - arr.mean(axis=0)` subtracts column means from every row — the mean (shape M) broadcasts across N rows. The rule: dimensions are compatible if they're equal or one of them is 1.

Q85 — How do you detect and remove duplicates in pandas?

`df.duplicated()` returns a boolean Series. `df.duplicated(subset=['col1', 'col2'])` checks specific columns. `df.drop_duplicates(keep='first')` keeps first occurrence. `keep='last'` for most recent.

Q86 — What is the difference between `copy()` and assignment in pandas?

`df2 = df` creates a reference — changes to df2 change df. `df2 = df.copy()` creates an independent copy. The `SettingWithCopyWarning` appears when pandas detects you might be modifying a copy of a slice without realising it.

Q87 — What does `df.info()` tell you?

Column names, non-null counts, dtypes, and memory usage. The most important quick-look method after loading data. Non-null count less than total rows = nulls present. Object dtype on a numeric column = data type problem.

Q88 — How do you pivot data in pandas?

`df.pivot_table(values='revenue', index='region', columns='quarter', aggfunc='sum', fill_value=0)` — cross-tabulation with aggregation. For raw pivoting without aggregation: `df.pivot(index='date', columns='product', values='revenue')` — fails if there are duplicate index/column pairs.

● Advanced Python — Q89 to Q100

🎯 These separate mid-level from senior. Expect these at FAANG and senior-level roles.

Q89 — What is a metaclass?

A class whose instances are classes. By default, all classes are instances of `type`. A metaclass controls how classes are created. `class Meta(type): ...` then `class MyClass(metaclass=Meta):`. Rare in application code — common in frameworks.

Q90 — What is `__slots__` and when is it worth using?

Replaces per-instance `__dict__` with a fixed descriptor. Reduces memory per instance — significant when creating millions of objects. Trade-off: you lose the ability to add arbitrary attributes. Use when memory is constrained and you know all attributes upfront.

Q91 — What is the difference between `threading` and `multiprocessing`?

`threading`: multiple threads, one process, shared memory, GIL prevents true CPU parallelism. Good for I/O-bound tasks. `multiprocessing`: multiple processes, separate memory, true CPU parallelism. Good for CPU-bound tasks. Inter-process communication is more complex.

Q92 — What is `async` / `await` in Python?

Coroutines for asynchronous programming. `async def` defines a coroutine. `await` suspends it until the awaited operation completes — releasing control for other coroutines. Uses a single thread. Good for I/O-bound work (APIs, DBs) with high concurrency. Requires an event loop: `asyncio.run(main())`.

Q93 — What is a descriptor?

An object that defines `__get__`, `__set__`, and/or `__delete__`. When accessed through a class or instance, Python calls these methods instead of returning the object directly. `@property` is a descriptor. Used to build `@property`, `@classmethod`, `@staticmethod`.

Q94 — What is `__call__`?

Makes an object callable like a function. When you do `obj()`, Python calls `obj.__call__()`. Useful for objects that need to maintain state between calls

while still being callable.

Q95 — What is the difference between `pickle` and `json` ?

`json` is text-based, human-readable, language-agnostic, but only handles basic Python types. `pickle` is binary, Python-specific, handles almost any Python object but is a security risk when loading untrusted data (it can execute arbitrary code).

Q96 — How does Python's `__new__` work vs `__init__` ?

`__new__` creates the object (allocates memory). `__init__` initialises it. You almost never override `__new__` — except for implementing singletons, immutable subclasses, or metaclass patterns. `__init__` doesn't create — it sets up.

Q97 — What is a singleton and how do you implement it?

A class that can only have one instance. Implement via `__new__` :

```
class Config:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Every call to `Config()` returns the same object.

Q98 — What is `lru_cache` and when do you use it?

`@functools.lru_cache(maxsize=128)` caches function results. When called with the same arguments again, returns the cached result instantly. Use for expensive pure functions — recursive Fibonacci, API calls, database lookups with repeated inputs. Only works on functions with hashable arguments.

Q99 — What is the difference between `__iter__` and `__next__` ?

`__iter__` returns the iterator object (often `self`). `__next__` returns the next value and raises `StopIteration` when done. A class implementing both is an iterator. A class implementing only `__iter__` (returning a separate iterator) is an iterable.

Q100 — What is a Python wheel (.whl file)?

A pre-built distribution format for Python packages. Installing a wheel is faster than installing from source because it doesn't need to compile. The alternative is a source distribution (`.tar.gz`). `pip install` prefers wheels when available. This comes up in deployment and DevOps-adjacent interviews.

● Quick Reference

Topic	Questions
Basics	Q01–Q20
Functions & Scope	Q21–Q35
Data Structures	Q36–Q50
OOP	Q51–Q65
Error Handling & Files	Q66–Q75
pandas & Data	Q76–Q88
Advanced Python	Q89–Q100